

Universidade Positivo

Modelagem Computacional

Validador Formal em Tres Niveis

Tema 1: Validador de Cadastro, Formulas e Integridade

Otavio Augusto Almeida dos Santos

RGM: 43873961

Repositorio: github.com/OtavioAugustoo/validador-formal

Junho de 2026

1 Introdução e Contexto

Toda vez que um software decide se uma entrada é válida, ele está, implicitamente, reconhecendo uma linguagem formal. Validadores de formulário, parsers de protocolo e verificadores de log operam sobre essa ideia sem necessariamente torná-la explícita.

Este projeto torna essa relação explícita, construindo três reconhecedores independentes aplicados a cada nível da hierarquia de Chomsky, aplicados ao mesmo domínio prático: validação de cadastro e integridade de dados.

Objetivo central: demonstrar empiricamente a hierarquia $LR \subsetneq LLC \subsetneq R$ por meio de implementação manual dos modelos computacionais DFA, PDA e Máquina de Turing em Python, sem uso do módulo `re` como mecanismo de reconhecimento.

1.1 Linguagens escolhidas

LR — Linguagem Regular: CPF no formato `ddd.ddd.ddd-dd` (11 dígitos separados por ponto e hífen). Decidível por DFA com poucos estados; não exige memória de contexto anterior.

LLC — Linguagem Livre de Contexto: Expressões com parênteses, colchetes e chaves balanceados. Exige memória ilimitada para rastrear profundidade de aninhamento, característica intrínseca de um PDA com pilha.

R — Linguagem Recursiva: $L = \{w\#w \mid w \in \{0,1\}^*\}$: cópia exata de cadeia binária separada por `#`. Exige comparação posicional entre duas metades, problema indecidível por autômatos finitos ou PDA; requer Máquina de Turing.

1.2 Hierarquia de poder computacional

A relação entre os três modelos é de contenção estrita:

$$LR \subsetneq LLC \subsetneq R$$

O CPF não pode ser reconhecido de forma genérica por qualquer DFA sem estados — ele exige exatamente 15 estados (`q0` a `q14` + `qerro`). O balanceamento de parênteses não pode ser decidido por nenhum DFA porque exige contar níveis. A cópia `w#w` não pode ser decidida por nenhum PDA porque exige comparação posição a posição entre as duas metades.

2 Nível 1 — Linguagem Regular (LR)

2.1 Descrição da linguagem

A linguagem reconhece CPFs no formato textual `ddd.ddd.ddd-dd`, em que `d` é qualquer dígito de 0 a 9. Valida-se apenas o formato — não os dígitos verificadores aritméticos.

2.2 Definição formal

$\Sigma = \{0,1,2,3,4,5,6,7,8,9, '.', '-' \}$

$L(DFA) = \{ w \in \Sigma^* \mid w \text{ tem exatamente 11 dígitos nos grupos 3.3.3-2} \}$

Expressão regular equivalente: `[0-9]{3}\.[0-9]{3}\.[0-9]{3}-[0-9]{2}` (apenas para referência — não usada no código)

2.3 Modelo computacional — DFA

O DFA possui 16 estados. Cada estado representa quantos símbolos corretos foram lidos até aquele momento. A tabela abaixo mostra as transições principais:

Estado	dígito	.	-	outro	É final?	Papel
q0	q1	qerro	qerro	qerro	Não	Início
q1	q2	qerro	qerro	qerro	Não	2º dígito
q2	q3	qerro	qerro	qerro	Não	3º dígito
q3	qerro	q4	qerro	qerro	Não	1º ponto
q4	q5	qerro	qerro	qerro	Não	4º dígito
q5	q6	qerro	qerro	qerro	Não	5º dígito
q6	q7	qerro	qerro	qerro	Não	6º dígito
q7	qerro	q8	qerro	qerro	Não	2º ponto
q8	q9	qerro	qerro	qerro	Não	7º dígito
q9	q10	qerro	qerro	qerro	Não	8º dígito
q10	q11	qerro	qerro	qerro	Não	9º dígito
q11	qerro	qerro	q12	qerro	Não	Hífen
q12	q13	qerro	qerro	qerro	Não	1º verif.
q13	q14	qerro	qerro	qerro	Não	2º verif.
q14	qerro	qerro	qerro	qerro	Sim ✓	Aceitação
qerro	qerro	qerro	qerro	qerro	Não	Estado morto

2.4 Exemplos de cadeias

Aceitas:

- `123.456.789-00` — formato exato, 14 símbolos, 14 passos
- `000.000.000-00` — dígitos zero, igualmente válido
- `999.999.999-99` — dígitos máximos

Rejeitadas:

- 12.345.678-90 — apenas 2 dígitos no primeiro grupo (rejeita no passo 3)
- 123456789-00 — falta o primeiro ponto (rejeita no passo 4)
- 123.456.789-000 — terceiro dígito verificador além do esperado (rejeita no passo 15)

2.5 Execução passo a passo — cadeia aceita

Cadeia: "123.456.789-00" — 14 símbolos — 14 passos

#	Estado	Símbolo	Próximo	Observação
1	q0	1	q1	1º dígito do grupo 1
2	q1	2	q2	2º dígito do grupo 1
3	q2	3	q3	3º dígito do grupo 1
4	q3	.	q4	1º separador (ponto)
5	q4	4	q5	1º dígito do grupo 2
6	q5	5	q6	2º dígito do grupo 2
7	q6	6	q7	3º dígito do grupo 2
8	q7	.	q8	2º separador (ponto)
9	q8	7	q9	1º dígito do grupo 3
10	q9	8	q10	2º dígito do grupo 3
11	q10	9	q11	3º dígito do grupo 3
12	q11	-	q12	Hífen separador
13	q12	0	q13	1º dígito verificador
14	q13	0	q14	2º dígito verificador ACEITA ✓

3 Nível 2 — Linguagem Livre de Contexto (LLC)

3.1 Descrição da linguagem

A linguagem reconhece expressões simbólicas com parênteses `()`, colchetes `[]` e chaves `{ }` balanceados e corretamente aninhados. Letras, números e operadores são ignorados pelo reconhecedor — apenas os agrupadores importam.

3.2 Definição formal

$\Sigma = \{ (,), [,], \{, \}, a-z, 0-9, +, -, *, /, \dots \}$

$\Gamma = \{ Z, (, [, \{ \}$ (alfabeto da pilha; Z é o marcador de fundo)

$L(PDA) = \{ w \in \Sigma^* \mid \text{os agrupadores de } w \text{ formam uma sequência balanceada} \}$

3.3 Gramática Livre de Contexto equivalente

Uma GLC que gera a mesma linguagem (ignorando outros símbolos):

$S: \varepsilon \mid S S \mid (S) \mid [S] \mid \{ S \}$

3.4 Modelo computacional — PDA

O PDA possui 3 estados: `q0` (leitura), `qf` (aceitação) e `qerro` (rejeição). A lógica central é simples: empilhar abre, desempilhar no fechamento correspondente.

Estado	Símbolo	Topo	Próximo	Ação (pilha)
q0	([{	qualquer	q0	Empilha o abre sobre o topo atual
q0	)] }	par correto	q0	Desempilha (pop do topo)
q0	)] }	par errado / Z	qerro	Rejeita (mismatch ou pilha vazia)
q0	(eps)	Z	qf	Cadeia acabou, pilha só tem Z ACEITA
q0	outros	qualquer	q0	Ignora (letras, números, operadores)

3.5 Exemplos de cadeias

Aceitas:

- `((x+y)*z)` — dois aninhamentos, 10 passos
- `{[a+b]}` — tipos diferentes aninhados, 8 passos
- `(a)` — caso mínimo, 4 passos

Rejeitadas:

- `((a+b)` — parêntese não fechado (pilha não vazia ao fim)
- `{[x+y]}` — mismatch: `}` fecha `[`, rejeita no passo 6
- `((())` — abre 3, fecha 2, pilha com 1 restante ao fim

3.6 Execução passo a passo — cadeia aceita e rejeitada

*Cadeia aceita: " $((x+y)^*z)$ " — 9 símbolos — 10 passos (inclui transição ϵ final)*

Passos: empilha "(" (p1), empilha "(" (p2), ignora x,+,y (p3-p5), desempilha "(" com ")" (p6), ignora * (p7), ignora z (p8), desempilha "(" com ")" (p9), transição ϵ para qf (p10).

Cadeia rejeitada: " $\{[x+y]\}$ " — rejeita no passo 6 por mismatch: ")" tentando fechar "["

4 Nível 3 — Linguagem Recursiva (R)

4.1 Descrição da linguagem

A linguagem reconhece cadeias do formato $w#w$, em que w é uma cadeia binária qualquer sobre $\{0,1\}$ e o símbolo $\#$ separa as duas cópias. Ambas as metades devem ser idênticas.

4.2 Definição formal

$\Sigma = \{0, 1, \#\}$

$\Gamma = \{0, 1, \#, X, Y, \sqcup\}$ (X = marca esquerda, Y = marca direita, $\sqcup$ = branco)

$L(MT) = \{w#w \mid w \in \{0,1\}^*\}$

Exemplos: $101\#101 \in L$; $0\#0 \in L$; $\# \notin L$; $101\#100 \notin L$

4.3 Por que é necessária uma MT

A linguagem $w#w$ não é livre de contexto. A prova informal: um PDA poderia empilhar a primeira metade, mas ao cruzar o $\#$ precisaria desempilhar na ordem correta. Como a pilha é LIFO, ele leria a segunda metade de trás pra frente, não da esquerda pra direita — impossível de fazer coincidir sem dois ponteiros independentes. A MT resolve isso com a cabeça leitora/escritora que pode varrer a fita em ambas as direções.

4.4 Algoritmo da Máquina de Turing

A MT usa 10 estados e o seguinte algoritmo iterativo:

1. Em q_0 , varre da esquerda procurando o primeiro 0 ou 1 não marcado
2. Marca com X e vai para q_1 (achou 0) ou q_2 (achou 1)
3. Atravessa toda a parte esquerda e o $\#$ até chegar na parte direita
4. Procura o símbolo correspondente (q_3 para 0, q_4 para 1); se encontrado, marca com Y
5. Volta para a esquerda (q_5 : q_6) e reinicia o scan
6. Quando q_0 encontra $\#$ (nenhum 0/1 restante na esquerda), entra em q_7
7. q_7 verifica se a parte direita está toda marcada com Y ; se sim, ACEITA

4.5 Tabela de transições (principais)

Estado	Lê	Escreve	Move	Próximo	Papel
q_0	0	X	R	q_1	Marca 0 na esquerda, vai buscar par
q_0	1	X	R	q_2	Marca 1 na esquerda, vai buscar par
q_0	X	X	R	q_0	Já marcado, pula
q_0	#	#	R	q_7	Esquerda toda marcada, verifica direita
q_1	0/1/X	=	R	q_1	Atravessa esquerda rumo ao #
q_1	#	#	R	q_3	Cruzou o #, procura 0 na direita

q3	0	Y	L	q5	Par correto! Marca com Y, volta
q3	1/␣	=	R	qrej	Mismatch ou sem par: rejeita
q2	#	#	R	q4	Cruzou o #, procura 1 na direita
q4	1	Y	L	q5	Par correto! Marca com Y, volta
q4	0/␣	=	R	qrej	Mismatch ou sem par: rejeita
q5	0/1/Y	=	L	q5	Volta pelo lado direito
q5	#	#	L	q6	Cruzou o # voltando
q6	0/1	=	L	q6	Volta pelo lado esquerdo
q6	X	X	R	q0	Chegou nos marcados, reinicia scan
q7	Y	Y	R	q7	Verifica direita toda marcada
q7	␣	␣	R	qacc	Tudo casado ACEITA
q7	0/1	=	R	qrej	Sobrou símbolo: rejeita

4.6 Exemplos de cadeias

Aceitas:

- 101#101 — 7 símbolos, 32 passos (3 iterações de marcação)
- 0#0 — 3 símbolos, 8 passos
- 11#11 — 5 símbolos, 18 passos

Rejeitadas:

- 101#100 — último bit diverge: $1 \neq 0$, rejeita no passo 23
- 1#10 — metades de tamanho diferente, rejeita no passo 8
- 0#1 — $0 \neq 1$, rejeita imediatamente no passo 3

5 Testes e Resultados

5.1 Metodologia

Cada reconhecedor foi testado com 6 cadeias (3 aceitas, 3 rejeitadas). O programa `src/testes.py` lê os arquivos em `testes/`, executa os três reconhecedores e imprime a tabela esperado × obtido com o número de passos.

Adicionalmente, o professor poderá testar cadeias surpresa compatíveis com o alfabeto e a definição formal de cada linguagem, sem que o reconhecedor precise ser alterado.

5.2 Linguagem Regular — DFA para CPF

Cadeia	Esperado	Obtido	Passos	Status
123.456.789-00	ACEITA	ACEITA	14	✓ Passou
000.000.000-00	ACEITA	ACEITA	14	✓ Passou
999.999.999-99	ACEITA	ACEITA	14	✓ Passou
12.345.678-90	REJEITA	REJEITA	3	✓ Passou
123.456.789-000	REJEITA	REJEITA	15	✓ Passou
123456789-00	REJEITA	REJEITA	4	✓ Passou

5.3 Linguagem Livre de Contexto — PDA para parênteses

Cadeia	Esperado	Obtido	Passos	Status
$((x+y)^*z)$	ACEITA	ACEITA	10	✓ Passou
$\{[a+b]\}$	ACEITA	ACEITA	8	✓ Passou
(a)	ACEITA	ACEITA	4	✓ Passou
$((a+b))$	REJEITA	REJEITA	6	✓ Passou
$\{[x+y]\}$	REJEITA	REJEITA	6	✓ Passou
$((()))$	REJEITA	REJEITA	5	✓ Passou

5.4 Linguagem Recursiva — MT para $L = \{w\#w\}$

Cadeia	Esperado	Obtido	Passos	Status
101#101	ACEITA	ACEITA	32	✓ Passou
0#0	ACEITA	ACEITA	8	✓ Passou
11#11	ACEITA	ACEITA	18	✓ Passou
101#100	REJEITA	REJEITA	23	✓ Passou
1#10	REJEITA	REJEITA	8	✓ Passou
0#1	REJEITA	REJEITA	3	✓ Passou

Total: 18/18 testes passaram.

6 Comparação entre os Três Níveis

6.1 Poder computacional

Os três modelos diferem fundamentalmente no que conseguem decidir:

Nível	Modelo	Linguagem	Cadeia (n=3)	Passos	Crescimento
LR	DFA	CPF (ddd.ddd.ddd-dd)	123.456.789-00	14	Linear $O(n)$
LLC	PDA	Parênteses balanceados	$((x+y)^*z)$	10	Linear $O(n)$
R	MT	$L = \{w\#w\}$	101#101	32	Quadrático $O(n^2)$

6.2 Crescimento do número de passos

O número de passos é a métrica mais clara da diferença de complexidade entre os níveis:

- **DFA (LR):** sempre exatamente n passos, onde n é o comprimento da cadeia. Para CPF: sempre 14 passos (ou menos se rejeitar cedo). Crescimento linear $O(n)$.
- **PDA (LLC):** também linear em n , pois cada símbolo gera no máximo 1 transição. A pilha cresce e decresce mas o número de passos é proporcional ao tamanho da entrada. Crescimento linear $O(n)$.
- **MT (R):** quadrático em relação ao tamanho de w . Para w de tamanho k , a MT realiza aproximadamente k^2 passos porque precisa varrer a fita inteira a cada marcação. "101#101" ($|w|=3$) gerou 32 passos; "0#0" ($|w|=1$) gerou 8 passos. Crescimento $O(n^2)$.

6.3 Memória utilizada

- **DFA:** memória finita e fixa — apenas o estado atual. Nenhuma estrutura de dados adicional.
- **PDA:** pilha potencialmente ilimitada — cresce com o nível de aninhamento da expressão.
- **MT:** fita potencialmente ilimitada em ambas as direções — cresce com o tamanho da cadeia e é usada para marcação iterativa.

6.4 Implicação prática

Na prática, essa hierarquia explica por que validadores de formulário (CPF, e-mail, IP) são simples e rápidos — operam no nível regular. Parsers de linguagens de programação (que precisam balancear blocos e expressões) operam no nível livre de contexto. Verificadores de protocolos com rastreamento de sequências completas (como espelhamento de transações) precisam do nível recursivo, pagando custo quadrático.

7 Implementação

7.1 Estrutura do repositório

projeto/

src/regular.py	— DFA para CPF (executável independente)
src/livre_contexto.py	— PDA para parênteses (executável independente)
src/recursiva.py	— MT para $w#w$ (executável independente)
src/testes.py	— runner que lê testes/*.txt e exibe tabelas
testes/	— 3 arquivos com cadeias aceitas e rejeitadas
diagramas/	— diagramas .dot e .png dos três autômatos
relatorio/	— este relatório em PDF

7.2 Decisões de implementação

- **Transições como dicionários:** os três reconhecedores declaram os estados, alfabetos, estado inicial, estados finais e função de transição explicitamente como dicionários/conjuntos Python, conforme exigido pelo enunciado.
- **Proibição do módulo re:** nenhum dos três arquivos usa `re` como mecanismo de aceitação/rejeição. O módulo não foi importado em nenhum reconhecedor.
- **Contagem de passos:** cada reconhecedor incrementa um contador exclusivamente nos eventos do modelo — leitura de símbolo com mudança de estado (DFA/NFA), transição com empilhamento/desempilhamento (PDA), e movimento da cabeça (MT).
- **Execução autônoma:** cada arquivo pode ser executado diretamente: `python src/regular.py "123.456.789-00"`.
- **Proteção contra loop infinito na MT:** limite de 100.000 passos para evitar execução infinita em entradas patológicas.

7.3 Como executar

Executar a bateria completa:

```
python src/testes.py
```

Executar um reconhecedor individualmente:

```
python src/regular.py "123.456.789-00"
```

```
python src/livre_contexto.py "((x+y)*z)"
```

```
python src/recursiva.py "101#101"
```

8 Conclusão e Limitações

8.1 Conclusão

O projeto demonstrou na prática a hierarquia $LR \subsetneq LLC \subsetneq R$. Os três reconhecedores foram implementados como simuladores manuais de autômatos — DFA, PDA e Máquina de Turing — e reconhecem corretamente 18/18 cadeias da bateria de testes.

A diferença de poder computacional ficou evidente tanto nos modelos utilizados quanto no número de passos: o DFA opera em tempo estritamente linear com memória constante; o PDA opera em tempo linear mas com memória proporcional ao aninhamento; a MT opera em tempo quadrático com memória proporcional à entrada.

Mais importante: cada nível reconhece estritamente mais linguagens que o anterior, e essa diferença não é apenas teórica — ela tem consequências diretas no design de sistemas reais.

8.2 Limitações

- O DFA para CPF valida apenas o formato textual, não os dígitos verificadores aritméticos (módulo 11).
- O PDA para parênteses ignora o conteúdo das expressões — valida apenas a estrutura de agrupamento.
- A MT para $w\#w$ suporta apenas alfabeto binário $\{0,1\}$; extensão para alfabetos maiores exigiria estados adicionais.
- A contagem de passos da MT cresce quadraticamente, o que a torna impraticável para cadeias muito longas.

github.com/OtavioAugustoo/validador-formal